Università degli Studi di Roma "Tor Vergata"
Laurea in Informatica

Sistemi Operativi e Reti
(modulo Reti)
a.a. 2025/2026

Livello di applicazione (parte 1)

dr. Manuel Fiorelli
manuel.fiorelli@uniroma2.it
https://art.uniroma2.it/fiorelli

Basate sulle slide del libro di testo:
https://gaia.cs.umass.edu/kurose_ross/ppt.php

Livello di applicazione: panoramica

- Principi delle applicazioni di rete
- Web e HTTP
- E-mail, SMTP, IMAP
- DNS: il servizio di directory di Internet
- Applicazioni P2P
- Streaming video e reti di distribuzione di contenti
- Programmazione delle socket programming con UDP e TCP



Livello di applicazione: panoramica

Obiettivi:

- Aspetti concettuali e implementativi dei protocolli delle applicazioni di rete
  - modelli di servizio del livello di trasporto
  - paradigma client-server
  - paradigma peer-to-peer
- apprendere informazioni sui protocolli esaminando i protocolli e l'infrastruttura delle più diffuse applicazioni di rete
  - HTTP
  - SMTP, IMAP
  - DNS
  - Sistemi per lo streaming video, CDNs
- programmare applicazioni di rete
  - socket API

Alcune diffuse applicazioni di rete

- posta elettronica (e-mail)
- trasferimento file
- newsgroup (es. Usenet) e poi forum
- Web
- messaggistica istantanea
- social media
- giochi multiutente via rete
- streaming di video-clip memorizzati (YouTube, Hulu, Netflix)
- condivisione di file P2P
- Applicazioni basate sulla posizione
- Applicazioni di pagamento mobile
- telefonia via Internet, voice-over-IP (VoIP), es. Skype
- videoconferenza in tempo reale
- ricerca sul Web
- shell o desktop remoto
- …

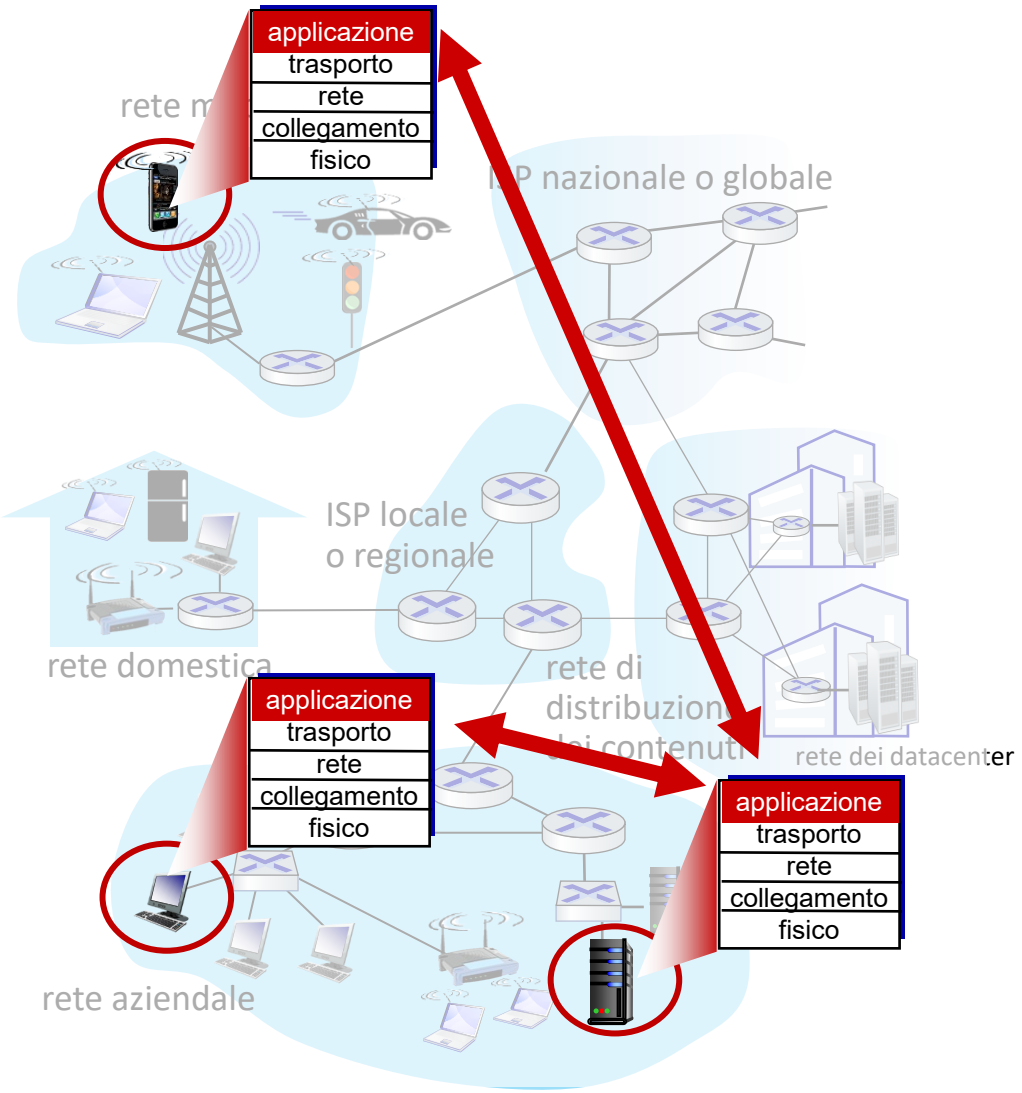
Creare un'applicazione di rete

scrivere programmi che:

- girano su sistemi periferici differenti
- comunicano attraverso la rete
- ad esempio, il software di un server Web comunica con il software di un browser

non è necessario scrivere programmi per i dispositivi nel nucleo della rete

- i dispositivi del nucleo della rete non eseguono applicazioni utente
- il confinamento delle applicazioni nei sistemi periferici ha facilitato il rapido sviluppo e la diffusione di una vasta gamma di applicazioni per Internet



Paradigma client-server

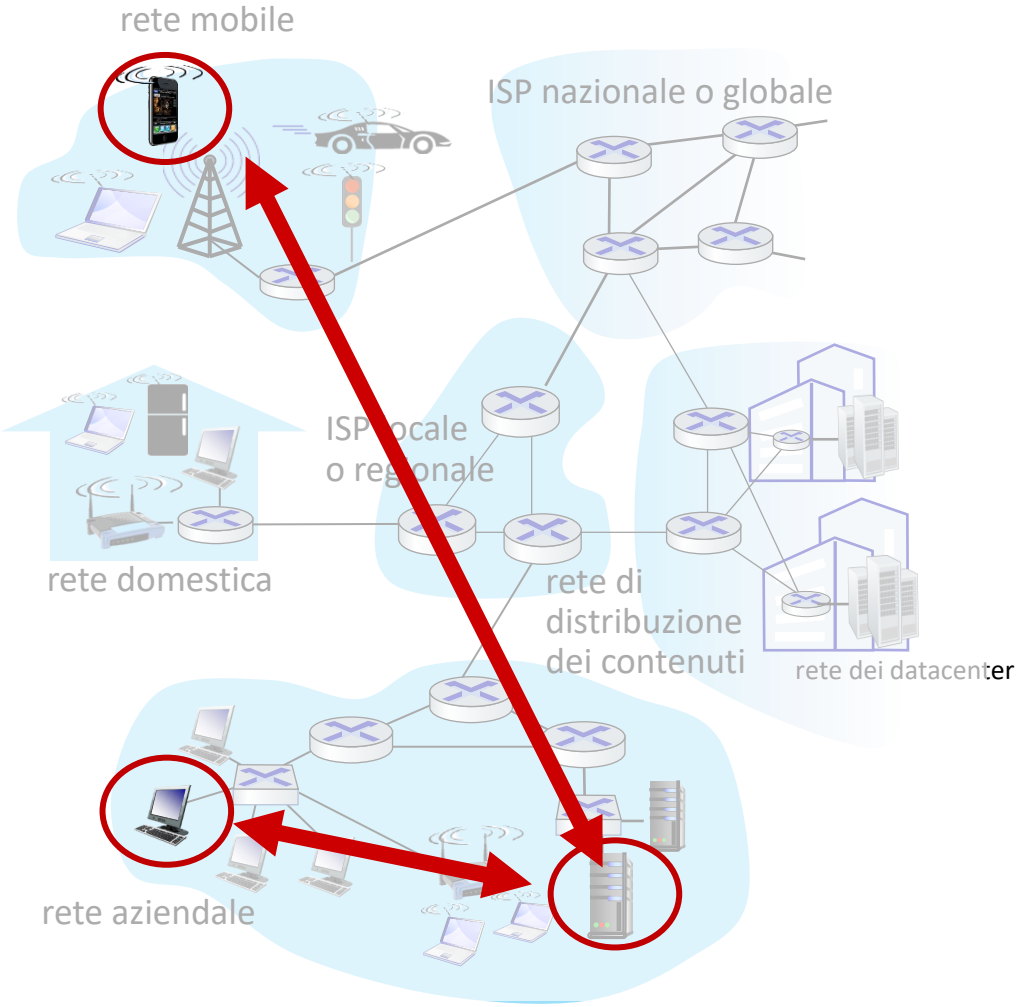
server:

- host sempre attivo
- indirizzo IP fisso
- spesso in datacenter, per la scalabilità
- onere economico sui fornitori del servizio (per infrastruttura, manutenzione, traffico)

client:

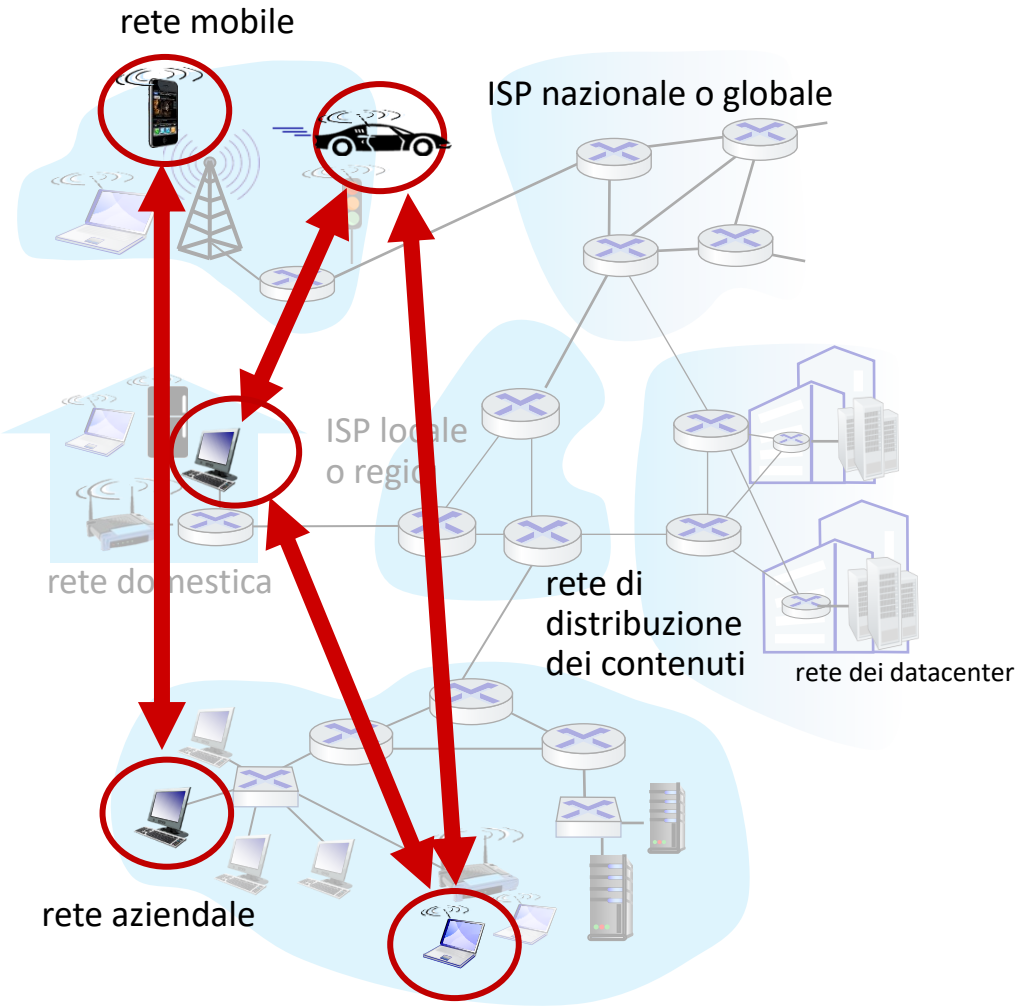
- contatta, comunica col server
- può contattare il server in qualunque momento
- può avere indirizzi IP dinamici
- *non* comunica direttamente con gli altri client

- esempi: Web, posta elettronica, etc.



Architettura peer-to-peer

- *non* c'è un server sempre attivo (o quantomeno, l'infrastruttura server è ridotta al minimo)
- coppie arbitrarie di host (peer) comunicano direttamente tra loro
- i peer richiedono un servizio ad altri peer e forniscono un servizio in cambio ad altri peer.
  - *scalabilità intrinseca* – nuovi peer aggiungono capacità di servizio al sistema, sebbene generino anche nuovo carico di lavoro
  - onere economico ridotto sui fornitori del servizio
- i peer non devono necessariamente essere sempre attivi e cambiano indirizzo IP
  - Difficile da gestire
- sfide riguardanti: prestazioni, affidabilità e sicurezza
- esempio: condivisione di file P2P [BitTorrent]



Processi comunicanti

processo: programma in esecuzione su di un host

- all'interno dello stesso host, due processi comunicano usando un approccio interprocesso (inter-process communication) (definito dal sistema operativo)
- processi su host differenti comunicano attraverso lo scambio di messaggi usando il servizio del livello di trasporto

client, server

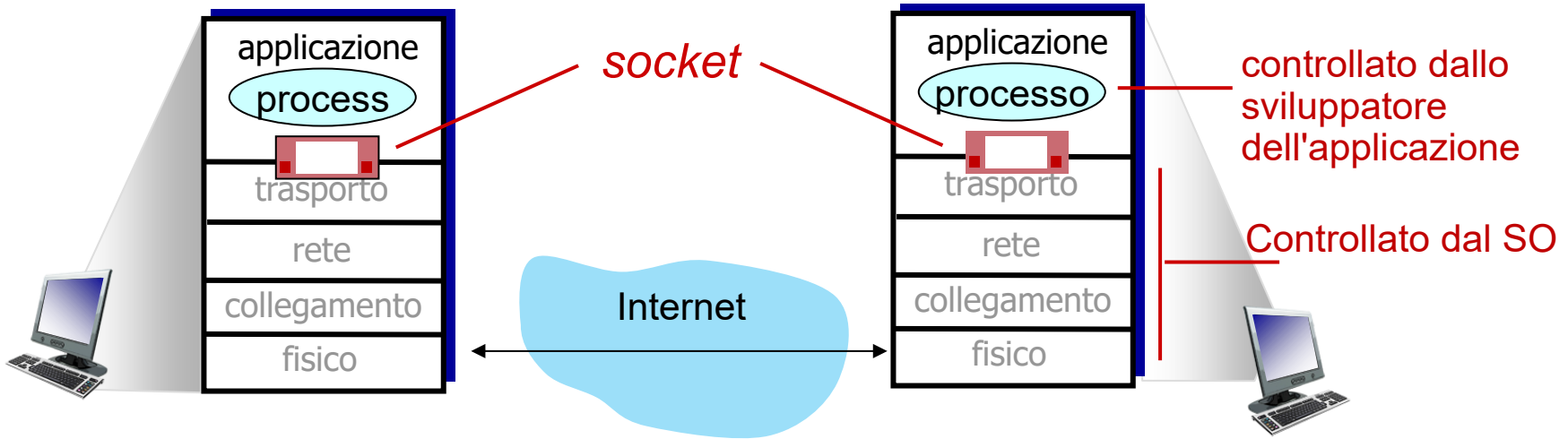
processo client: processo che dà inizio alla comunicazione (in altri termini, richiede il servizio/l'informazione)

processo server: processo che attende di essere contattato (in altri termini fornisce il servizio/l'informazione)

- nota: nelle applicazioni P2P, un processo può essere sia client sia server; ciononostante, nel contesto di una specifica sessione di comunicazione, ogni processo adotterà l'uno o l'altro ruolo

Socket

- un processo invia/riceve messaggi a/da la sua socket
- una socket è analoga a una porta
  - Il processo mittente fa uscire il messaggio fuori dalla propria "porta" (socket)
  - Il processo mittente presuppone l'esistenza di un'infrastruttura esterna che traporterà il messaggio attraverso la rete fino alla "porta" del processo di destinazione



Indirizzamento

- per ricevere messaggi, un processo deve avere un *identificatore*
- Un host ha un indirizzo IP univoco a 32 bit (IPv4) o 128 bit (IPv6)
- *D:* è sufficiente conoscere l'indirizzo IP dell'host su cui è in esecuzione un process per identificare il processo stesso?
- *R:* no, sullo stesso host possono essere in esecuzione *molti* processi.

- *L'identificatore* comprende sia l'indirizzo IP (IP address) che i numeri di porta (port number) associati al processo in esecuzione su un host.
- numeri di porta assegnati (dallo IANA) a applicazioni più note:
  - HTTP server: 80
  - mail server: 25
- Per inviare un messaggio HTTP al server gaia.cs.umass.edu :
  - indirizzo IP: 128.119.245.12
  - numero di porta: 80

Un protocollo a livello applicazione definisce

- tipi di messaggi scambiati,
  - es, richiesta, risposta
- sintassi dei messaggi:
  - quali sono i campi nel messaggio e come sono descritti
- semantica dei messaggi
  - significato delle informazioni nei campi
- regole per determinare quando e come un processo invia e risponde ai messaggi

protocolli di pubblico dominio:

- definiti nelle RFC, ognuno ha accesso alla definizione
- consentono l'interoperabilità
- ad esempio, HTTP, SMTP

protocolli proprietari:

- ad esempio, Skype, Zoom

Quale servizio di trasposto richiede un'applicazione?

perdita di dati

- alcune applicazioni (es: trasferimento file, transazioni web) richiedono un trasferimento 100% affidabile
- altre applicazioni (es: applicazioni multimediali audio/video) possono tollerare qualche perdita

sensibilità al fattore tempo

- alcune applicazioni (ad esempio, telefonia via Internet, giochi interattivi) richiedono che i ritardi siano bassi per essere “efficaci”

throughput

- alcune applicazioni (dette "sensibili alla banda"), ad esempio, quelle multimediali, per essere efficaci richiedono un'ampiezza di banda minima- (Es., il throughput medio non deve essere inferiore al bitrate per la codifica audio/video)
- altre applicazioni (le “applicazioni elastiche”) utilizzano l'ampiezza di banda che si rende disponibile. (Es., lo scaricamento di un file può procedere nonostante la riduzione della banda disponibile, impiegando però più tempo)

sicurezza

- riservatezza, integrità dei dati, autenticazione, …

Requisiti del servizio di trasporto di alcune applicazioni comuni

applicazione	tolleranza alla perdita di dati	throughput	sensibilità al fattore tempo
trasferimento file	no	variabile	no
posta elettronica	no	variabile	no
documenti Web	no	variabile	no
audio/video in tempo reale	sì	audio: 5kbps-1Mbps video:10kbps-5Mbps	sì, centinaia di ms
streaming audio/video memorizzati	Sì	come sopra	sì, pochi secondi
giochi interattivi	sì	fino a pochi kbps	sì, centinaia di ms
messaggistica istantanea	no	variabile	sì e no

Servizi dei protocolli di trasporto di Internet

Servizio di TCP:

- *trasporto affidabile* fra i processi di invio e di ricezione: dati consegnati senza errori, perdite e nell'ordine di invio
- *controllo di flusso:* il mittente non vuole sovraccaricare il destinatario
- *controllo della congestione:* "strozza" il processo d'invio quando la rete è sovraccarica
- *orientato alla connessione:* è richiesto un setup fra i processi client e server
- *non offre:* temporizzazione, garanzie su un'ampiezza di banda minima, sicurezza

Servizio di UDP:

- *trasferimento dai inaffidabile* fra i processi d'invio e di ricezione
- *non offre:* affidabilità, controllo di flusso, controllo della congestione, temporizzazione, ampiezza di banda minima, sicurezza, né setup della connessione.

D: perché preoccuparsi?
Perché c'è UDP?

R: UDP offre alle applicazioni maggiore controllo sull'invio dei dati (senza garanzie sul throughput end-to-end disponibile, né ritardi)

Applicazioni Internet: protocollo a livello applicazione e protocollo di trasporto

applicazione	protocollo a livello applicazione	Protocollo di trasporto sottostante
trasferimento file	FTP [RFC 959]	TCP
posta elettronica	SMTP [RFC 5321]	TCP
documenti web	HTTP [RFC 7230, 9110]	TCP
telefonia via Internet	SIP [RFC 3261], RTP [RFC 3550], o proprietario	UDP o TCP
streaming audio/video memorizzati	HTTP [RFC 7230], DASH	TCP
giochi interattivi	WOW, FPS (proprietario)	UDP o TCP

TCP previsto spesso come fallback a causa di firewall che bloccano UDP

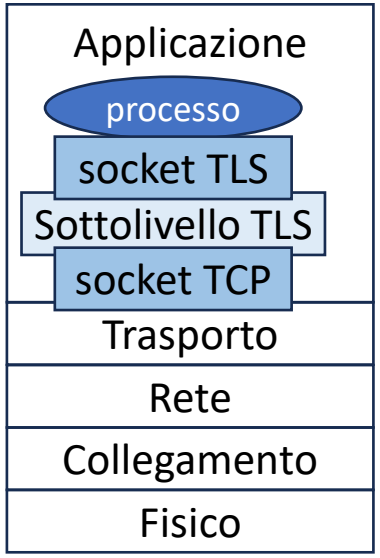
Rendere sicuro TCP

Socket TCP e UDP:

- nessuna cifratura
- password inviate in chiaro, cioè senza cifratura, attraverso socket attraversano Internet in chiaro(!)

Transport Layer Security (TLS)

- offre connessioni TCP cifrate
- controllo di integrità dei dati
- autenticazione end-to-end



TLS implementato a livello applicazione

- le applicazioni usano librerie TLS, che usano a propria volta TCP
- testo in chiaro (*cleartext*) inviato nella "socket TLS" viene *cifrato* prima di essere immesso nella socket TCP e attraversare Internet fino a destinazione; qui, i dati (cifrati) consegnati dalla socket TCP sono decifrati prima di essere restituiti in chiaro dalla socket TLS

Livello applicazione: panoramica

- Principi delle applicazioni di rete
- Web e HTTP
- E-mail, SMTP, IMAP
- DNS: il servizio di directory di Internet
- Applicazioni P2P
- Streaming video e reti di distribuzione di contenti
- Programmazione delle socket programming con UDP e TCP



Web e HTTP

*gli oggetti sono in realtà chiamati *risorse*

**lo *schema* indica il protocollo usato per accedere alla risorsa; altri esempi, sono *https* per HTTP sopra TLS o *ftp* per l'accesso tramite FTP

Ripassiamo la terminologia…

- una pagina web è costituita da *oggetti*,* ciascuno dei quali può essere memorizzato su un server Web differente
- un oggetto può essere un file HTML, un'immagine JPEG, uno script JavaScript, un foglio di stile CSS, un file audio, …
- una pagina web è formata da *un file HTML di base* che include *diversi oggetti referenziati, ciascuno* referenziato da un *URL*, ad esempio,

`http://www.someschool.edu/someDept/pic.gif`

schema** (*scheme*) | nome dell'host (*host name*) | percorso (*path name*)

Panoramica su HTTP

HTTP: HyperText Transfer Protocol

- protocollo a livello applicazione del Web
- modello client/server:
  - *client:* browser che richiede, riceve (usando il protocollo HTTP) e "visualizza" gli oggetti del Web
  - *server:* il server Web che invia (usando il protocollo HTTP) oggetti in risposta alle richieste



Panoramica su HTTP (continua)

HTTP usa TCP (fino a HTTP/3 escluso):

- il client inizializza la connessione TCP (crea una socket) con il server, sulla porta 80
- il server accetta la connessione TCP dal client
- messaggi HTTP (messaggi di un protocollo di applicazione) scambiati tra browser (client HTTP) e server Web (server HTTP)
- connessione chiusa TCP

HTTP è un protocollo "senza stato" (stateless)

- Il server non mantiene informazioni sulle richieste fatte dal client

nota

i protocolli che mantengono lo stato sono complessi!

- la storia passato (stato) deve essere memorizzata
- se il server e/o il client si bloccano, le loro viste dello "stato" potrebbero essere contrastanti e dovrebbero essere riconciliate

Connessioni HTTP: due tipi

Connessioni non persistenti

1. connessione TCP aperta
2. una singola coppia richiesta/risposta HTTP per un singolo oggetto
3. connessione TCP chiusa

Lo scaricamento di oggetti multipli richiede connessioni multiple

Connessioni persistenti

- connessione TCP al server aperta
- più coppie richiesta/risposta HTTP, per trasmettere *più oggetti* su una *singola connessione* TCP tra client e server
- connessione TCP chiusa

Connessioni non persistenti

L'utente immette l'URL: **`http://www.someSchool.edu/someDepartment/home.html`**
(contiene testo, riferimenti a 10 immagini JPEG)



1a. Il client HTTP client inizializza una connessione TCP con il server HTTP (processo) a www.someSchool.edu sulla porta 80



1b. Il server HTTP all'host www.someSchool.edu in attesa di una connessione TCP alla porta 80 "accetta" la connessione e avvisa il client

2. Il client HTTP trasmette *un messaggio di richiesta* HTTP (contenente l'URL) nella socket della connessione TCP. Il messaggio indica che il client vuole l'oggetto someDepartment/home.html

3. Il server HTTP riceve il messaggio di richiesta, forma il *messaggio di risposta* che contiene l'oggetto richiesto e invia il messaggio nella sua socket

tempo

Connessioni non persistenti (continua)

L'utente immette l'URL: `http://www.someSchool.edu/someDepartment/home.html`
(contiene testo, riferimenti a 10 immagini JPEG)





4. Il server HTTP chiude la connessione TCP

5. Il client HTTP riceve il messaggio di risposta che contiene il file HTML e visualizza il document HTML. Esamina il file HTML, trova I riferimenti a 10 oggetti JPEG

6. I passi 1-5 sono ripetuti per ciascuno dei 10 oggetti JPEG

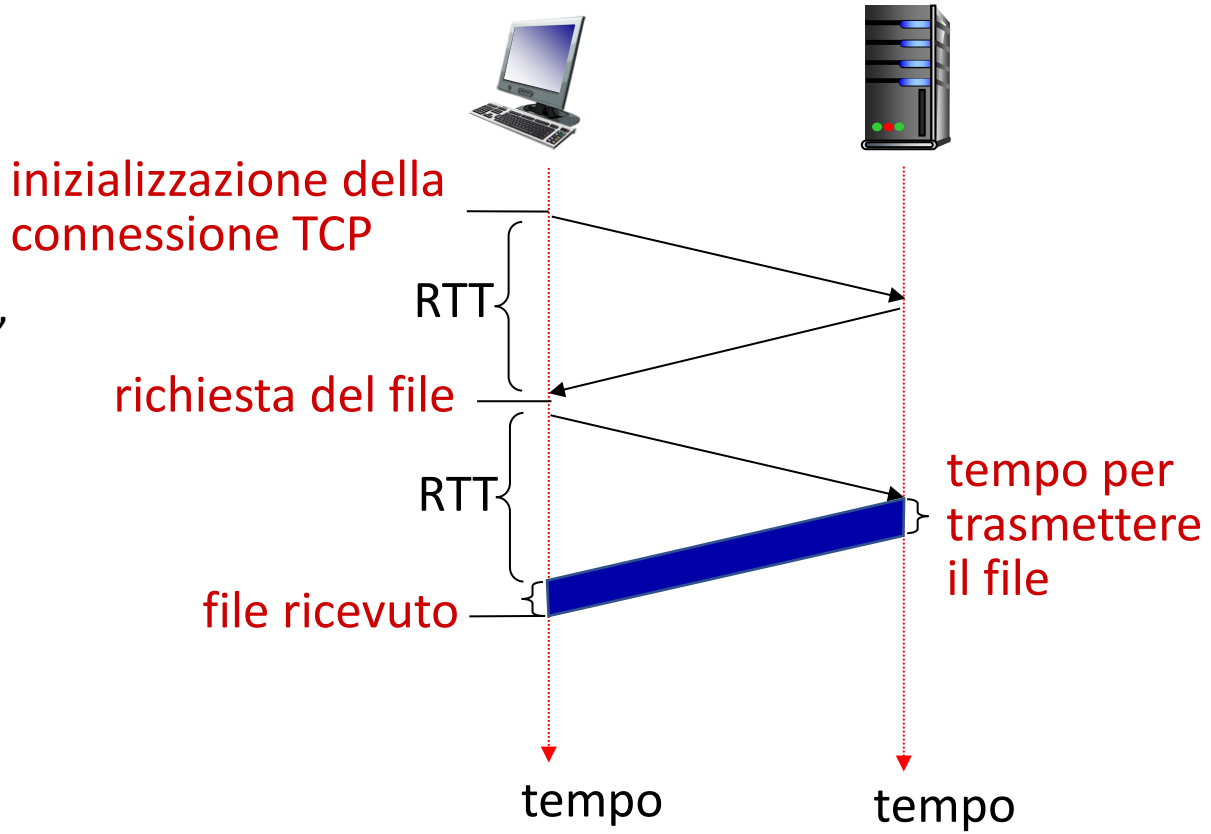
time

Connessioni non persistenti: tempo di risposta

RTT (definizione): tempo impiegato da un piccolo pacchetto per andare dal client al server e ritornare al client (include ritardi di elaborazione, accodamento, propagazione)

Tempo di risposta (per oggetto):

- un RTT per inizializzare la connessione TCP
- un RTT perché ritornino la richiesta HTTP e i primi byte della risposta HTTP
- tempo di trasmissione del file/oggetto



Tempo di risposta con connessioni non persistenti = 2RTT+ tempo di trasmissione del file

Connessioni persistenti (HTTP 1.1)

Svantaggi delle connessioni non persistenti:

- richiedono 2 RTT per oggetto
- allocazione di buffer e variabili per *ogni* connessione TCP
- i browser spesso aprono connessioni TCP parallele per caricare gli oggetti referenziati
  - maggiore onere sul server web
  - controllo della congestione: throughput inizialmente limitato per nuove connessioni (MA connessioni multiple permettono di aggirare la suddivisione equa della banda tra le applicazioni)

Connessioni persistenti (HTTP1.1):

- il server lascia la connessione TCP aperta dopo l'invio di una risposta
- i successivi messaggi tra gli stessi client/server vengono trasmessi sulla connessione aperta
- il client invia le richieste non appena incontra un oggetto referenziato
- un solo RTT per tutti gli oggetti referenziati

Messaggio di richiesta HTTP

- Due tipi di messaggi HTTP: *richiesta, risposta*
- Messaggio di richiesta HTTP:
  - ASCII (formato leggibile dall'utente)

riga di richiesta (*request line*) (comandi GET, POST, HEAD)

carattere di nuova linea (*line-feed*)

righe di intestazione (*header lines*)

```
GET /index.html HTTP/1.1\r\n
Host: www-net.cs.umass.edu\r\n
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X
   10.15; rv:80.0) Gecko/20100101 Firefox/80.0 \r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Connection: keep-alive\r\n
\r\n
```

Un carriage return e un, line feed all'inizio della linea indicano la fine delle righe di intestazione

* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

Messaggio di richiesta HTTP (continua)

Alcuni campi di intestazione nei messaggi di richiesta:

Host

- hostname e numero di porta (se assente, si assume 80 per HTTP e 443 per HTTPS) del server al quale sarà inviata la richiesta. Obbligatorio in HTTP/1.1; se assente, il server può rispondere con un 400 Bad Request. Necessario per il funzionamento delle web cache e del name-based virtual hosting.

User-Agent

- Identifica l'applicazione, il sistema operativo, il *vendor* e/o la versione dello *user agent* che sta effettuando la richiesta

Accept

- Tipi di contento, espressi come medita type, compresi dal client

Accept-Language

- Linguaggi naturali o *locale* preferiti dal client

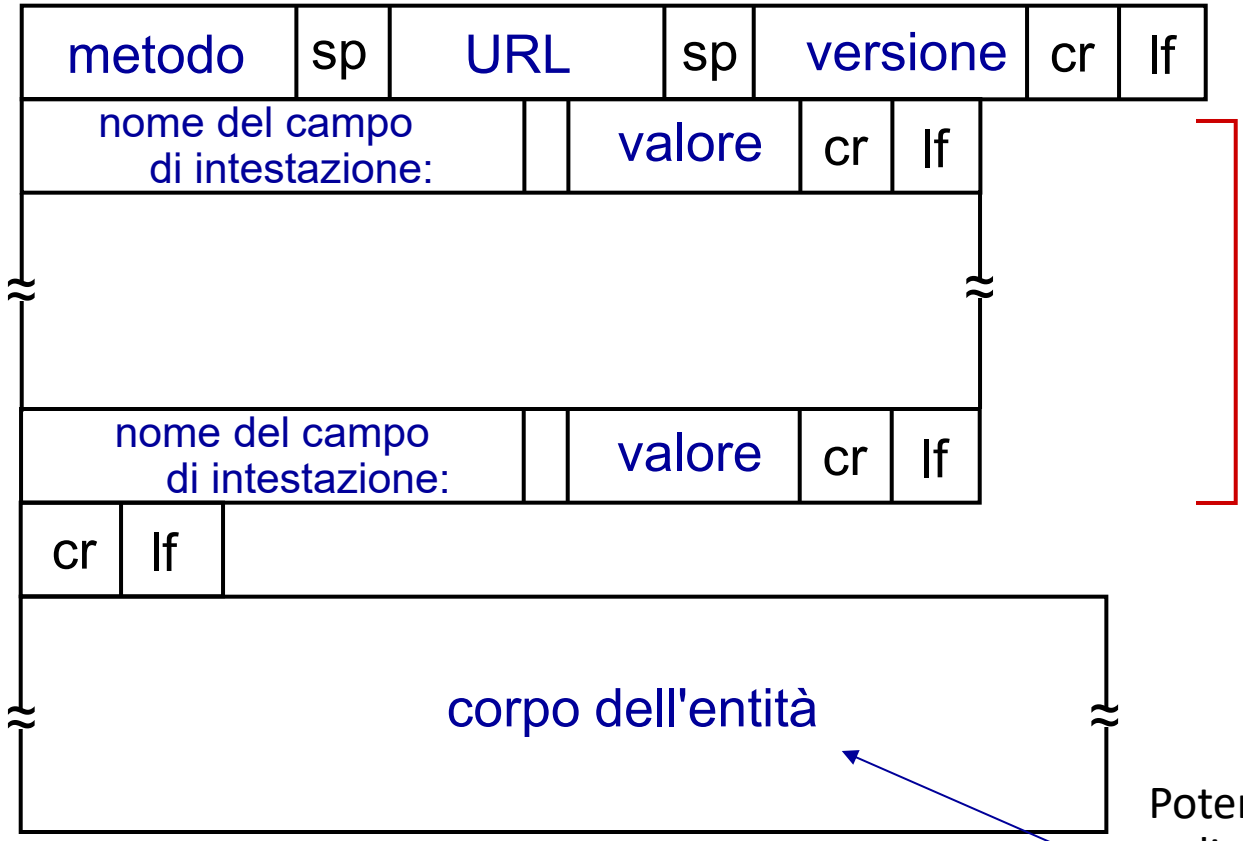
Accept-Encoding

- Algoritmi di codifica (tipicamente, la compressione) compresi dal client

Connection

- Controlla se la connessione rimarrà aperta al termine dello scambio richiesta/risposta. Il valore *close* indica che la connessione sarà chiusa (default in HTTP/1.0); altrimenti, una lista non vuota di nomi di header (in genere solo keep-alive), che saranno rimossi dal primo proxy non trasparente o cache, indica che la connessione rimarrà aperta (default in HTTP/1.1)

Messaggio di richiesta HTTP: formato generale



riga di richiesta (*request line*)

righe di intestazione (*header lines*)

entità (*body*)

Potenzialmente binario (es. un'immagine); non usa un terminatore come \r\n, ma l'intestazione ne fornisce la lunghezza nel campo *Content-Length*.

Altri messaggi di richiesta HTTP

Metodo POST:

- la pagina web spesso include un form per l'input dell'utente
- l'input dell'utente viene inviato dal client al server nel corpo dell'entità di un messaggio di richiesta HTTP POST

Metodo GET (per inviare dati al server):

- l'input arriva al server nel campo URL della riga di richiesta (dopo un '?'):

```
http://www.somesite.com
/search?q=monkey&kingdom=animals
```

Metodo HEAD:

- richiede le intestazioni (solo) che verrebbero restituite se l'URL specificato fosse richiesto con il metodo HTTP GET.

Metodo PUT:

- carica un nuovo file (oggetto) sul server
- sostituisce completamente il file esistente all'URL specificato con il contenuto del corpo dell'entità del messaggio di richiesta HTTP PUT.

HTTP: Idempotenza

Un'operazione si dice *idempotente* se l'effetto inteso sul server di una singola richiesta è lo stesso di quello di più richieste identiche

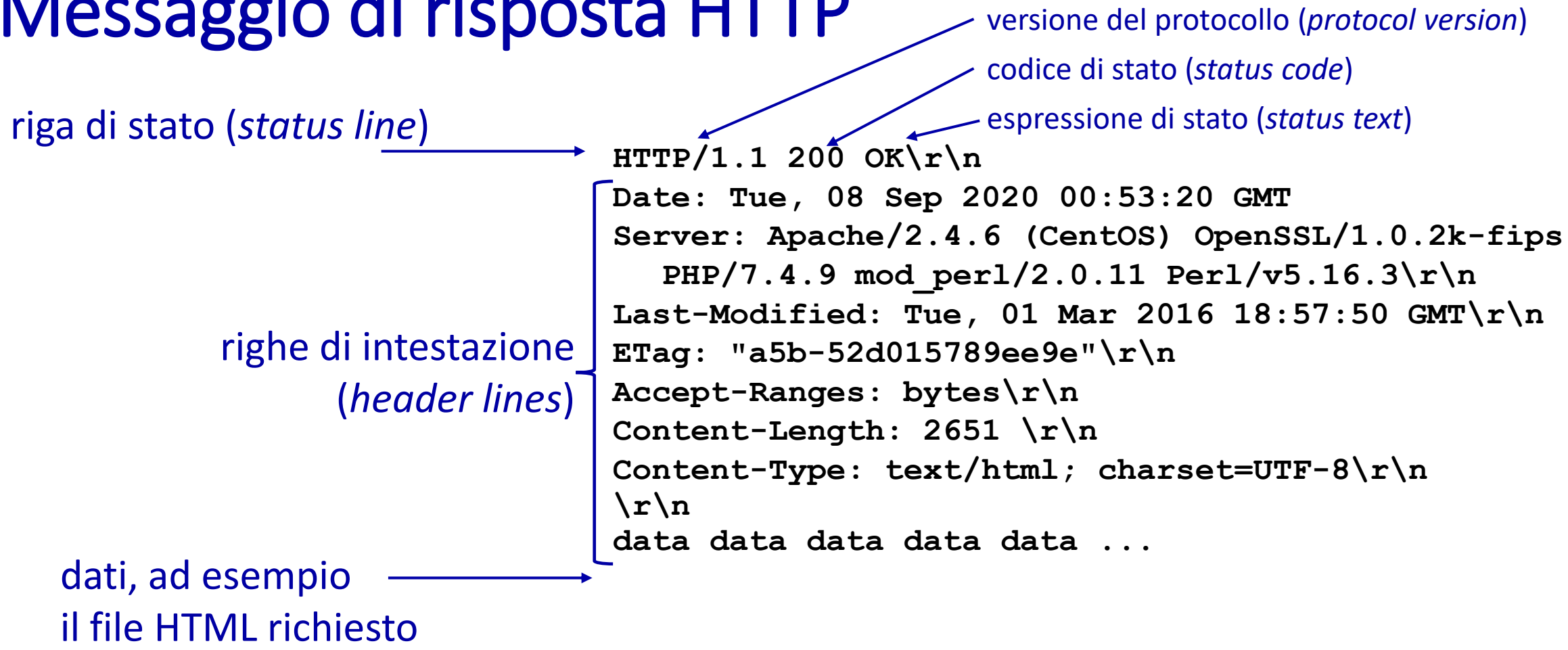
- i metodi GET, PUT, HEAD sono idempotenti
- il metodo POST non è idempotente (quantomeno non in generale)

Un form può essere implementato usando:

- GET: se l'operazione associata al form è idempotente (es. ricerca)
- POST: se l'operazione associata al form NON è idempotente (es. aggiunta di un prodotto al carrello di un e-commerce)

Il concetto di idempotenza è utile per stabilire se un client può *ritentare* in automatico una richiesta in caso di problemi di rete (transitori).

Messaggio di risposta HTTP



* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

Messaggio di risposta HTTP (continua)

Alcuni campi di intestazione nei messaggi di richiesta:

Date
- la data e l'ora in cui il messaggio è stato originato

Server
- descrive il software usato dal server di origine per gestire la richiesta (nota: troppi dettagli posso aiutare i malintenzionati a attaccare il server)

Last-Modified
- la data e l'ora in cui il server di origine crede che l'oggetto sia stato modificato per l'ultima volta

Accept-Ranges
- Indica il supporto del server ai download parziali: il valore, se diverso da none, indica l'unità che si può usare per esprimere l'intervallo richiesto

Content-Length
- lunghezza in byte del corpo dell'entità inviato al ricevente (o che *sarebbe* stato inviato nel caso di un richiesta HEAD)

Content-Type
- *media type* (che indica un formato) del corpo dell'entità inviato al ricevente (o che *sarebbe* stato inviato nel caso di un richiesta HEAD)

Codici di stato della risposta HTTP

- Nella prima riga nel messaggio di risposta dal server al client
- Definiti da RFC 9110
- Raggruppati in cinque categorie, discriminate dalla prima cifra

1xx Informational

- una risposta intermedia per comunicare lo stato di connessione o l'avanzamento della richiesta prima di completare l'azione richiesta e inviare una risposta finale (assenti in HTTP/1.0)

2xx Successful

- La richiesta è stata ricevuta con successo, compresa e accettata

3xx Redirect

- Il client deve eseguire ulteriori azioni per soddisfare la richiesta

4xx Client Error

- La richiesta è sintatticamente scorretta o non può essere soddisfatta

5xx Server Error

- Il server ha fallito nel soddisfare una richiesta apparentemente valida

Codici di stato della risposta HTTP

- Nella prima riga nel messaggio di risposta dal server al client
- Alcuni codici di stato e relative espressioni

200 OK
- La richiesta ha avuto successo; l'oggetto richiesto viene inviato nella risposta

301 Moved Permanently
- L'oggetto richiesto è stato trasferito; la nuova posizione è specificata nell'intestazione Location: della risposta

400 Bad Request
- Il messaggio di richiesta non è stato compreso dal server

404 Not Found
- Il documento richiesto non si trova su questo server

406 Not Acceptable
- L'oggetto richiesto non esiste in una forma che soddisfa i vari Accept-*

505 HTTP Version Not Supported
- Il server non ha la versione di protocollo HTTP

Provate HTTP (lato client)

1. Collegatevi via netcat al vostro server web preferito:

   % nc -c -v gaia.cs.umass.edu 80 (for Mac)

   >ncat –C gaia.cs.umass.edu 80 (for Windows)

   - apre una connessione TCP alla porta 80 (porta di default per un server HTTP) dell'host gaia.cs.umass.edu.
   - tutto ciò che digitate viene trasmesso alla porta 80 di gaia.cs.umass.edu

2. type in a GET HTTP request:

```
GET /kurose_ross/interactive/index.php HTTP/1.1
Host: gaia.cs.umass.edu
```

   - digitando questo (premete due volte il tasto Invio),trasmettete una richiesta GET minima (ma completa) al server HTTP

3. guardate il messaggio di risposta trasmesso dal server HTTP!

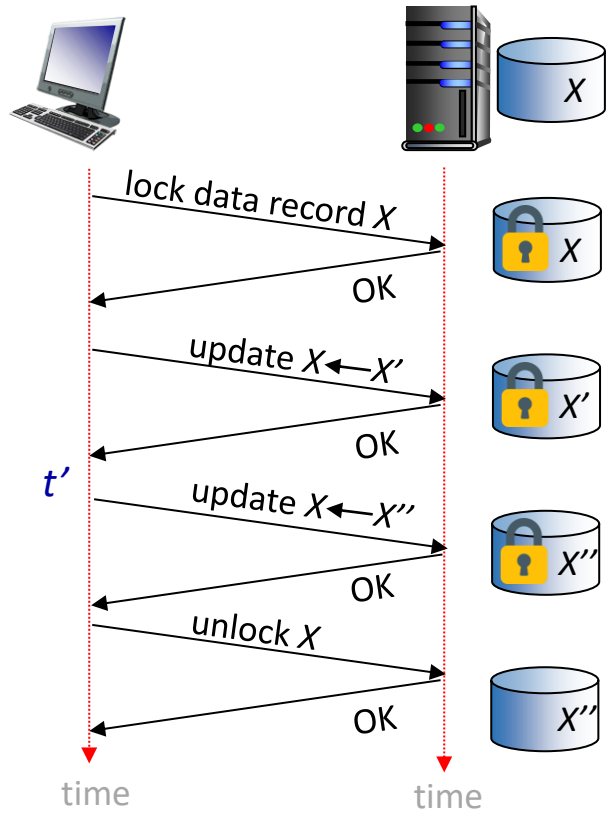
   (oppure usate Wireshark per guardare la richiesta e la risposta HTTP catturate)

Mantenere stato utente/server: i cookie

Ricordate: l'interazione HTTP GET/risposta è *senza stato (stateless)*

- nessuna nozione di scambio di messaggi HTTP in più fasi per completare una "transazione" Web
  - non è necessario che il client o il server tengano traccia dello "stato" dello scambio in più fasi
  - tutte le richieste HTTP sono indipendenti l'una dall'altra
  - non è necessario che il client né il server siano in grado di “recuperare” da una transazione quasi completa ma mai completata

un protocollo con stato: il client fa due modifiche a X, o nessuna



Q: che succede se la connessione di rete o il client si blocca al tempo *t'* ?

Mantenere stato utente/server: i cookie

I siti web e il browser client usano i *cookie* per mantener dello stato tra le transazioni

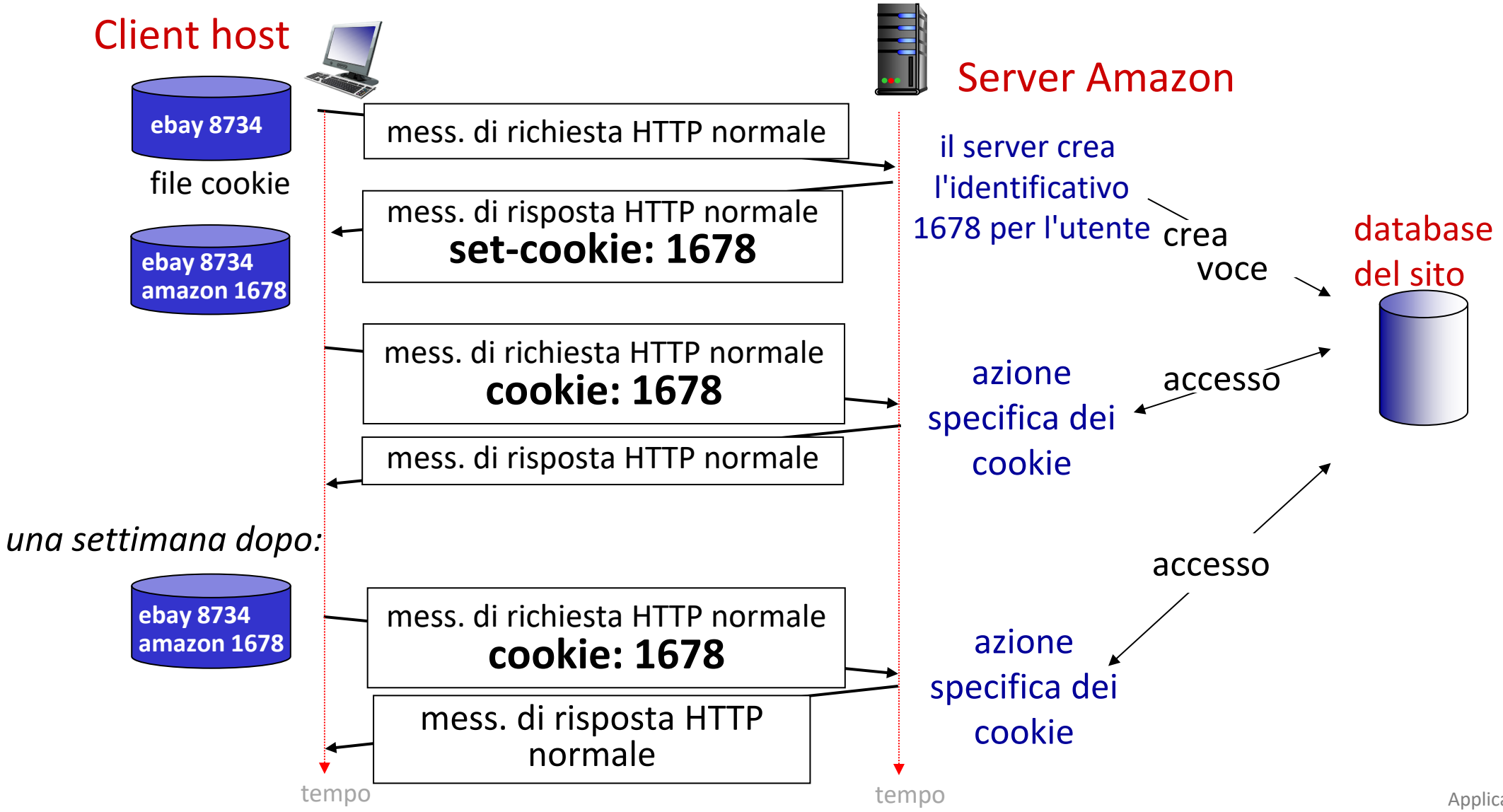
quattro componenti:

1) una riga di intestazione nel messaggio di *risposta* HTTP
2) una riga di intestazione nel messaggio di *richiesta* HTTP
3) un file cookie mantenuto sul sistema terminale dell'utente e gestito dal browser dell'utente
4) un database sul sito

Esempio:

- Susan usa il browser dal portatile, visita uno specifico siti di commercio elettronico per la prima volta
- quando la richiesta HTTP iniziale arriva al sito, il sito crea:
  - un identificativo unico
  - una voce nel proprio database, indicizzata dal numero identificativo
- Il server ritorna una risposta che include l'intestazione Set-cookie, che contiene l'identificativo unico e che sarà aggiunto al file dei cookie
- le successive richieste del browser di Susan per questo sito conterranno l'identificativo in una intestazione cookie

Mantenere stato utente/server: i cookie



Cookie HTTP: commenti

I cookie possono essere usati per:

- autorizzazione
- carrello degli acquisti
- raccomandazioni
- stato della sessione dell'utente (e-mail Web)

Sfida: Come mantenere lo stato?

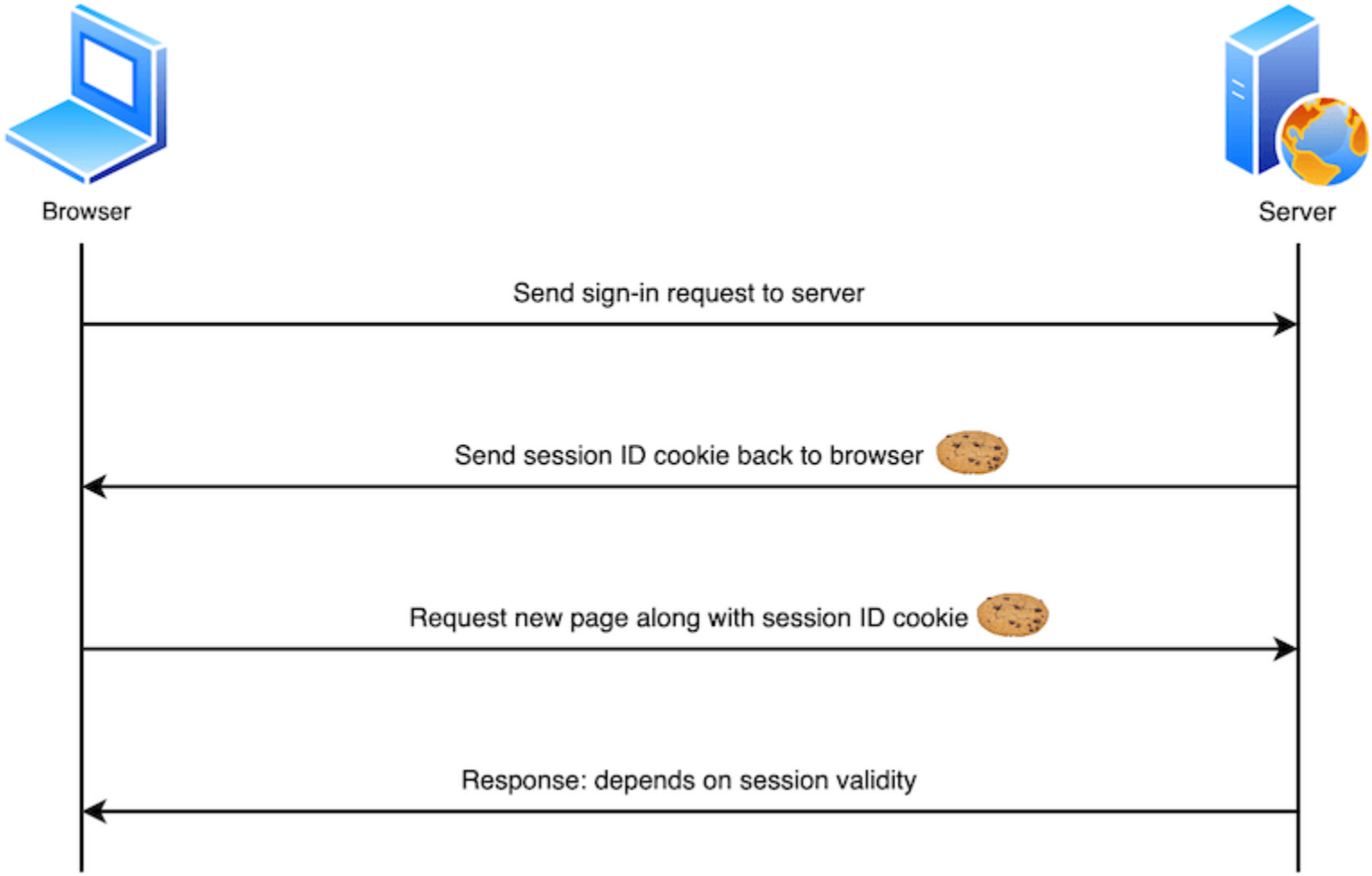
- *presso gli endpoint del protocollo:* mantenere lo stato presso il trasmettitore e il ricevitore attraverso multiple transazioni
- *nei messaggi:* i cookie trasportano lo stato nei messaggi HTTP

nota

cookie e privacy:

- I cookie consentono ai siti di *imparare* molto di voi
- cookie persistenti di terze parti (cookie di tracciamento, *tracking cookies*) consentono il tracciamento di una identità comune (valore del cookie) attraverso siti web multipli

Esempio: autorizzazione



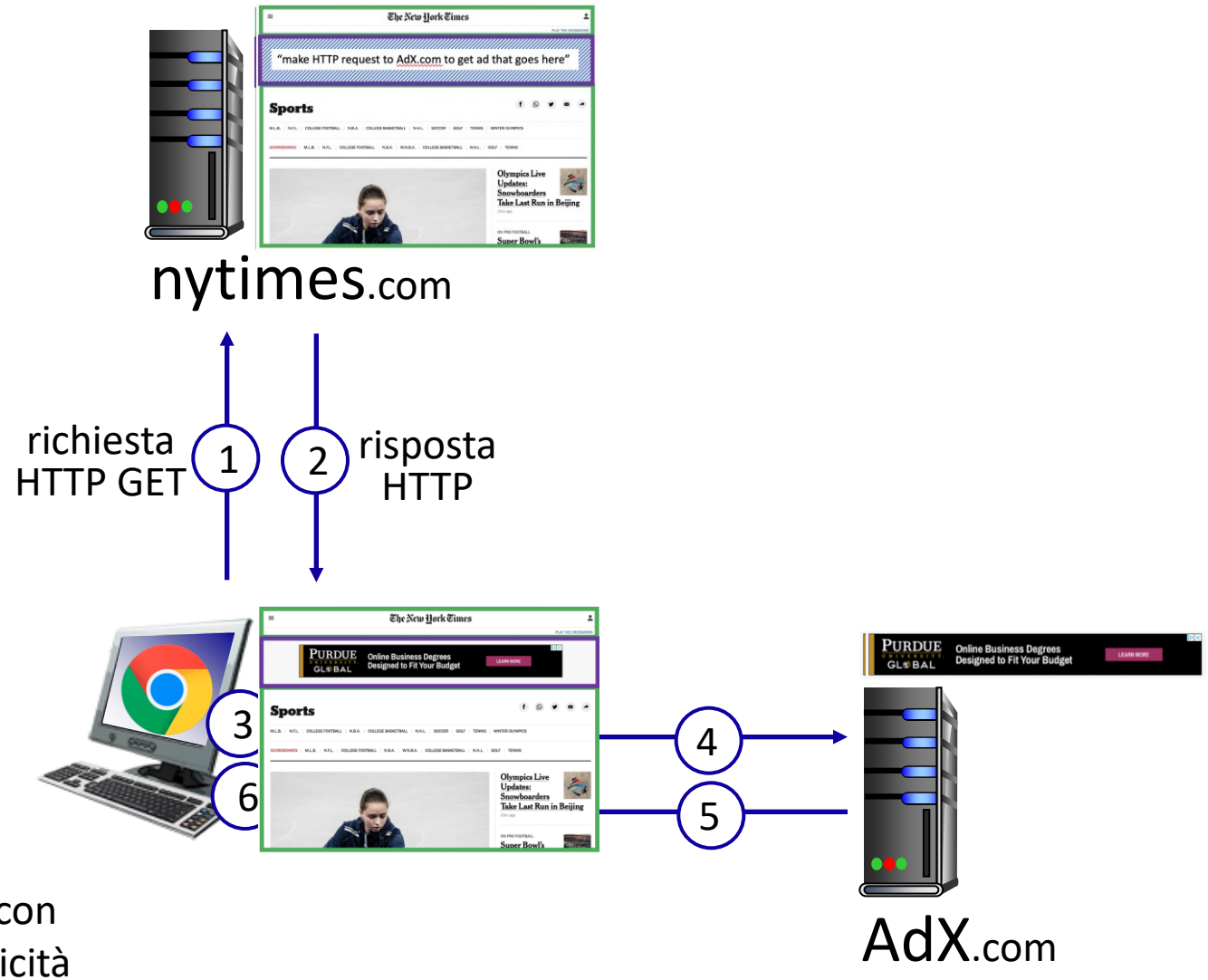
Fonte immagine: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Cookies

Esempio: visualizzare una pagina web del NY Times

1 2 invia una GET per il file HTML di base da nytimes.com

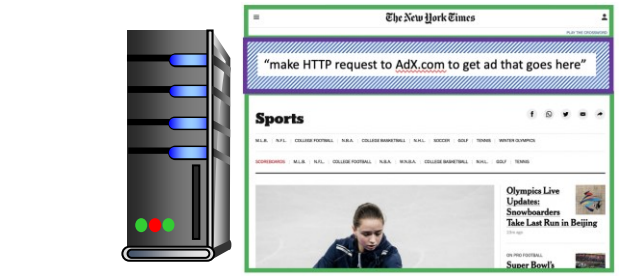
4 5 recupera l'annuncio da AdX.com

7 mostra la pagina composta

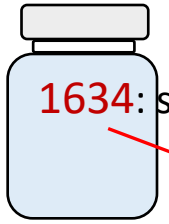


pagina del NY con mostrate pubblicità incorporate

Cookie: tracciare il comportamento di navigazione di un utente



nytimes.com (sports)



1634: sports, 2/15/22

HTTP GET

risposta HTTP
Set cookie: 1634

Cookie di "prima parte" ("first party") – da siti che hai scelto di visitare (fornisce il file HTML di base)

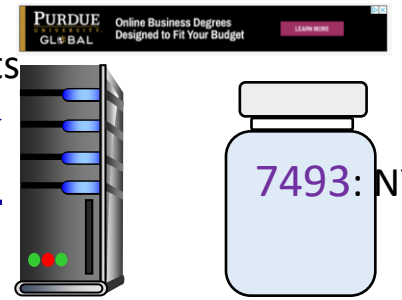


HTTP GET
Referrer: NY Times Sports

4

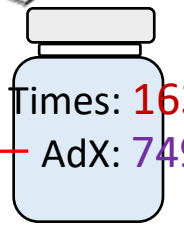
5

HTTP reply
Set cookie: 7493



7493: NY Times sports, 2/15/22

AdX.com



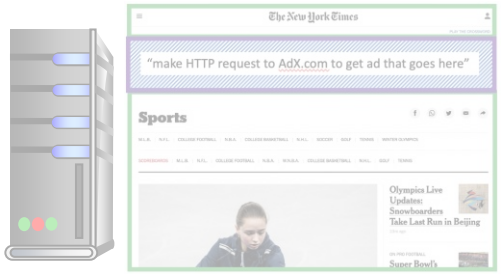
NY Times: 1634
AdX: 7493

cookie di "terze parti" ("third party") cookie – da siti web che <u>non</u> hai scelto di visitare

Cookie: tracciare il comportamento di navigazione di un utente



socks.com



nytimes.com



HTTP reply 2

HTTP GET 1



HTTP GET
Referrer: socks.com, cookie: 7493

4

5

HTTP reply







AdX.com

AdX:

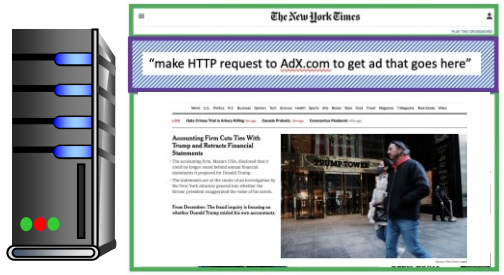
- *traccia la mia navigazione* su siti con annunci AdX
- può restituire annunci mirati in base alla cronologia di navigazione

Cookie: tracciare il comportamento di navigazione di un utente

(un giorno dopo)



socks.com



nytimes.com (arts)

1634: sports, 2/15/22
1634: arts, 2/17/22

HTTP GET
cookie: 1634

HTTP reply



HTTP GET
Referrer: nytimes.com, cookie: 7493

4

5

HTTP reply

NY Times: 1634
AdX: 7493

Restituito annuncio per I calzini!

AdX.com

7493: NY Times sports, 2/15/22
7493: socks.com, 2/16/22
7493: NY Times arts, 2/17/22

Cookie: tracciare il comportamento di navigazione di un utente

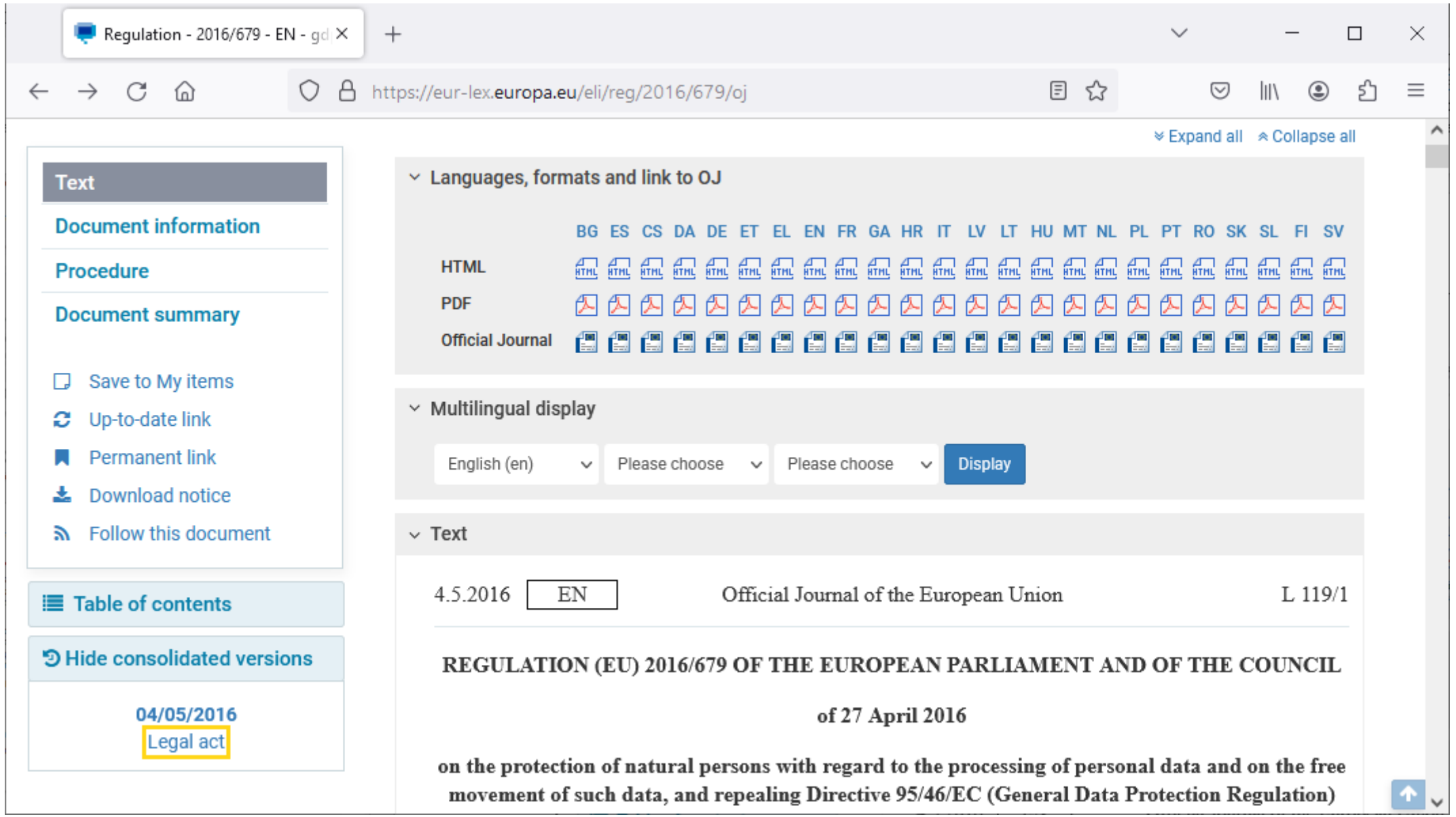
I cookie posso essere usati per:

- tracciare (*track*) il comportamento degli utenti su un dato sito (cookie di prima parte)
- tracciare il comportamento degli utenti su più siti (cookie di terze parti) senza neppure che l'utente abbia mai scelto di visitare il sito del tracker (!)
- il tracciamento può essere *invisibile* all'utente:
  - piuttosto che un annuncio visualizzato che attiva HTTP GET al tracker, potrebbe essere un collegamento invisibile

tracciamento di terze parti tramite cookie:

- disabilitato per impostazione predefinita nei browser Firefox e Safari
- eliminazione graduale dei cookie di terze parti nel browser Chrome, inizialmente bloccati per l'1% degli utenti a partire da Gennaio 2024, con l'obiettivo di estendere il blocco a tutti nel terzo trimestre del 2024

GDPR (EU General Data Protection Regulation)



GDPR (EU General Data Protection Regulation) e i cookie

"Le persone fisiche possono essere associate a identificativi online [..], quali gli indirizzi IP, a marcatori temporanei (cookies) o a identificativi di altro tipo, [...].

Tali identificativi possono lasciare tracce che, in particolare se combinate con identificativi univoci e altre informazioni ricevute dai server, possono essere utilizzate per creare profili delle persone fisiche e identificarle.."

GDPR, recital 30 (May 2018)

quando i cookie possono identificare un individuo, i cookie sono considerati dati personali, soggetti alla normativa GDPR sui dati personali

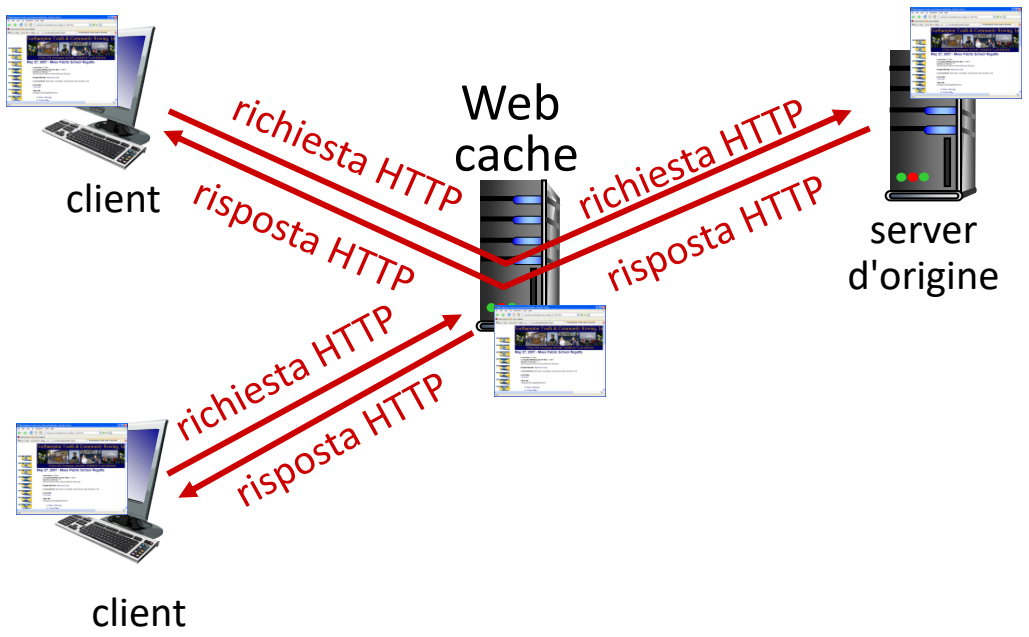


L'utente ha un controllo esplicito sull'autorizzazione o meno dei cookie.

Web cache

Obiettivo: soddisfare la richiesta del client senza coinvolgere il server d'origine (*origin server*)

- l'utente configura il browser per usare una *Web cache* (locale)
- Il browser trasmette tutte le richieste HTTP alla cache
  - *se* l'oggetto è nella cache: la cache fornisce l'oggetto al client
  - *altrimenti* la cache richiede l'oggetto al server d'origine, memorizza ("cache") l'oggetto ricevuto, e infine lo restituisce al client



la cache opera come client (per il server d'origine) e come server (per il client originale)

Web cache (server proxy)

Esempio di risposta del server d'origine (consente cache per 1 ora)

Il campo `Date` indica l'istante (data e orario) in cui il messaggio è stato generato, da non confondere con il valore del campo `Last-Modified`

La direttiva max-age (nel campo Cache-Control) indica per quanti secondi (a partire dal valore di `Date`) la risposta può essere considerata *fresh* e quindi usata da una potenziale cache per soddisfare le richieste dei client.

Nota: sono possibili *stale response* se la cache non riesce a contattare il server d'origine oppure se ciò è consentito dal server e/o client (tramite direttive)

```
HTTP/1.1 200 OK
Date: Sun, 15 Mar 2026 20:00:00 GMT
Content-Type: text/html; charset=utf-8
Cache-Control: max-age=3600, must-revalidate
Content-Length: 191

<!DOCTYPE html>
<html lang="it">
[…]
```

Indica che la cache non può restituire stale reponse

Web cache (server proxy)

Esempio di risposta del server d'origine (richiede validazione)

```
HTTP/1.1 200 OK
Date: Sun, 15 Mar 2026 20:00:00 GMT
Content-Type: text/html; charset=utf-8
Cache-Control: no-cache
Content-Length: 191

<!DOCTYPE html>
<html lang="it">
[…]
```

La direttiva `no-cache` (nel campo Cache-Control) non significa che la risposta possa essere memorizzata, ma solo che non può essere usata senza essere rivalidata (si veda GET condizionale…)

Web cache (server proxy)

Il campo di intestazione `Cache-Control` può essere usato anche nelle richieste:

- `Cache-Control: max-age=3600`

  La direttiva max-age viene usata dal client per indicare di volere una risposta generata entro il numero di secondi indicato. Non riceverò *stale response*, a meno che ciò non venga consentito da altre direttive (max-stale)

- `Cache-Control: no-cache`

  Il client non vuole un copia memorizzata in una cache senza che questa sia stata validata presso il server di origine.

GET condizionale (per la validazione della cache)

Obiettivo: non inviare un oggetto se la cache ha una copia aggiornata dell'oggetto

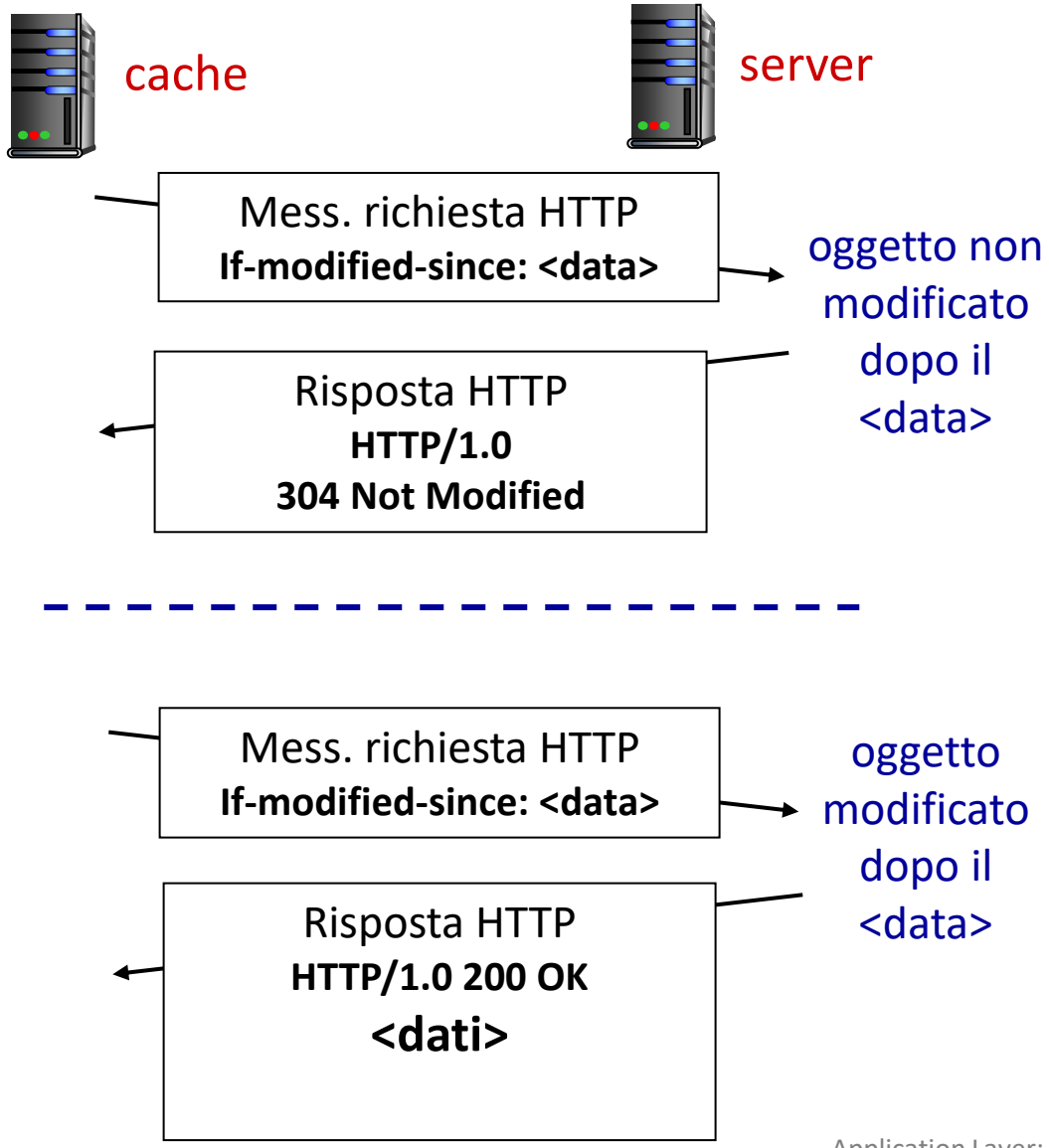
- Nessun ritardo di trasmissione dell'oggetto (o uso delle risorse di rete)

- *client:* specifica la data della copia dell'oggetto nella richiesta HTTP

  If-modified-since: <data>

- *server:* la risposta non contiene l'oggetto se la copia nella cache è aggiornata:

  HTTP/1.0 304 Not Modified



Web cache (server proxy)

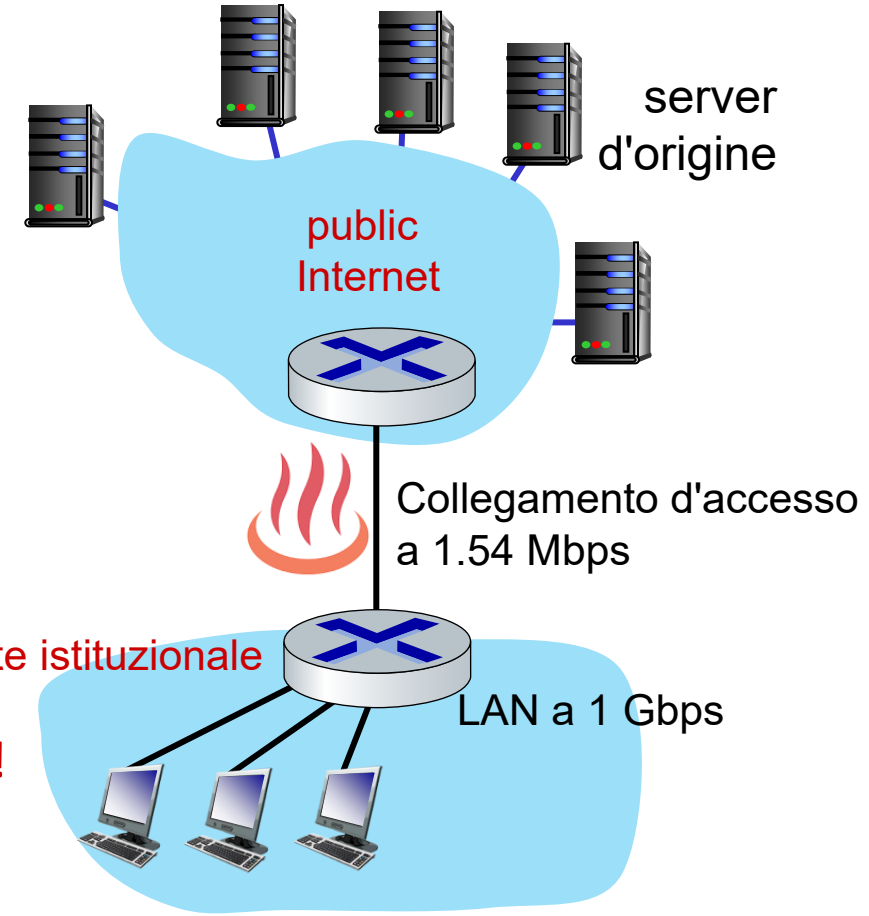
Perché il web caching?

- riduce i tempi di risposta alle richieste dei client
  - la cache è più vicina ai client
  - banda disponibile tra client e cache maggiore che nel percorso verso il server di origine (es., evita di attraversare il collegamento di accesso a Internet che spesso è il collo di bottiglia)
- riduce il traffico sul collegamento di accesso a Internet istituzionale
- Riduce il traffico globale su Internet
  - beneficia tutte le applicazioni

Esempio di caching

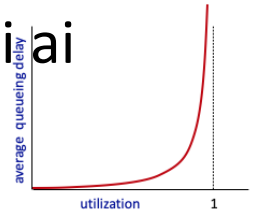
Scenario:

- velocità collegamento d'accesso: 1.54 Mbps
- ritardo di Internet (da router lato Internet al server): 2 s
- dimensione di un oggetto: 100 kbit
- frequenza media di richieste dai browser istituzionali al server d'origine: 15/s
  - velocità media di trasmissione dei dati ai browser: 1.50 Mbps



Prestazioni:

- utilizzazione del collegamento d'accesso = .97
- utilizzazione della LAN: .0015
- end-end delay = ritardo di Internet + ritardo del collegamento d'accesso + ritardo della LAN
  = 2 s + minuti + microsecondi



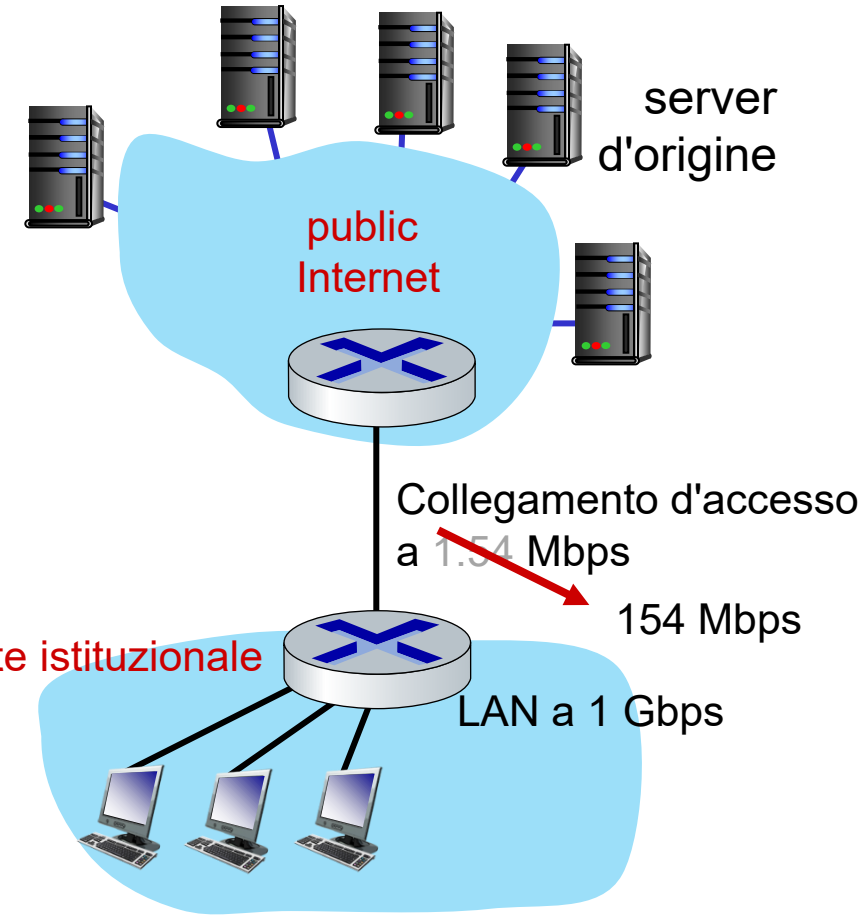
problema: ritardo d'accodamento elevato con elevata utilizzazione!

Opzione 1: collegamento d'accesso più veloce

Scenario:

- velocità collegamento d'accesso: 1.54 Mbps
- ritardo di Internet (da router lato Internet al server): 2 s
- dimensione di un oggetto: 100 kbit
- frequenza media di richieste dai browser istituzionali al server d'origine: 15/s
  - velocità media di trasmissione dei dati ai browser: 1.50 Mbps

154 Mbps



Prestazioni:

- utilizzazione del collegamento d'accesso = .97 → .0097
- utilizzazione della LAN: .0015
- end-end delay = ritardo di Internet + ritardo del collegamento d'accesso + ritardo della LAN
  = 2 s + minuti + microsecondi

millisecondi

Costo: collegamento d'accesso più veloce (costoso!)

Opzione 2: installare un web cache

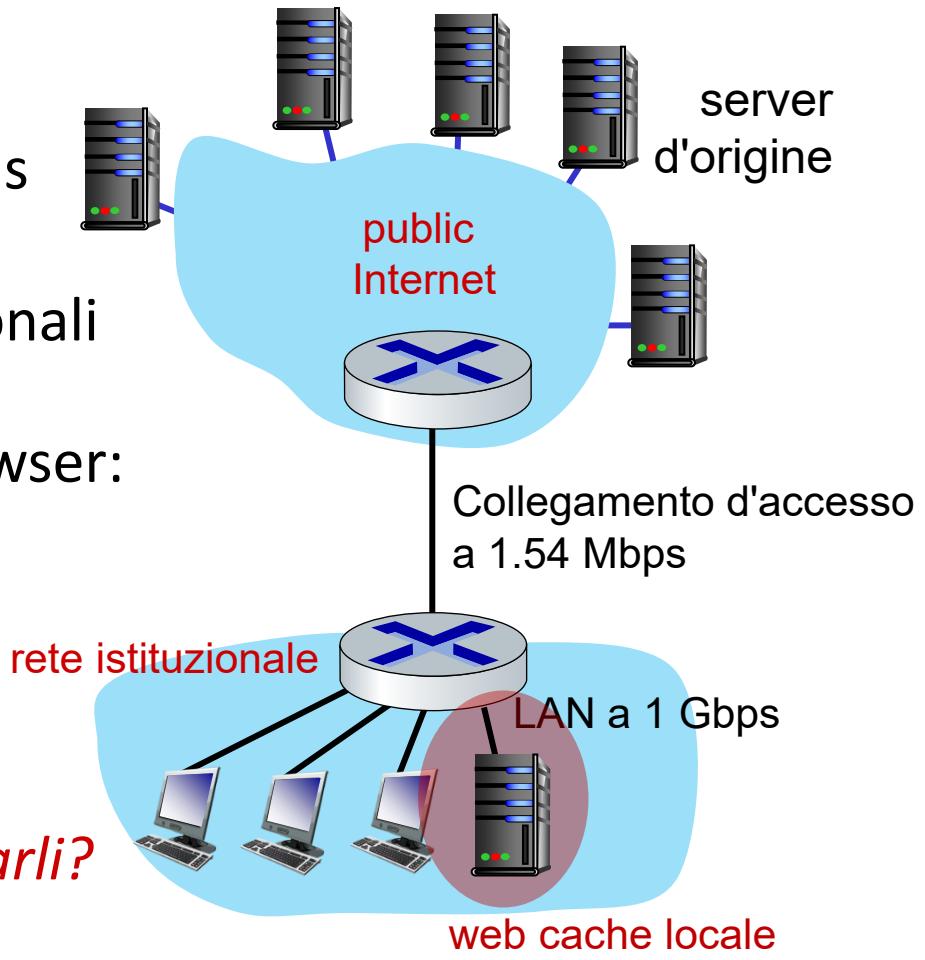
Scenario:

- velocità collegamento d'accesso: 1.54 Mbps
- ritardo di Internet (da router lato Internet al server): 2 s
- dimensione di un oggetto: 100 kbit
- frequenza media di richieste dai browser istituzionali al server d'origine: 15/s
  - velocità media di trasmissione dei dati ai browser: 1.50 Mbps

Costo: web cache (economica!)

Prestazioni:

- utilizzazione LAN: .?
- utilizzazione del link di accesso = ? *come calcolarli?*
- ritardo end-end medio = ?



Calcolo dell'utilizzo del collegamento di accesso e del ritardo end-end con la cache:

supponiamo una percentuale di successo (hit rate) pari a 0.4:

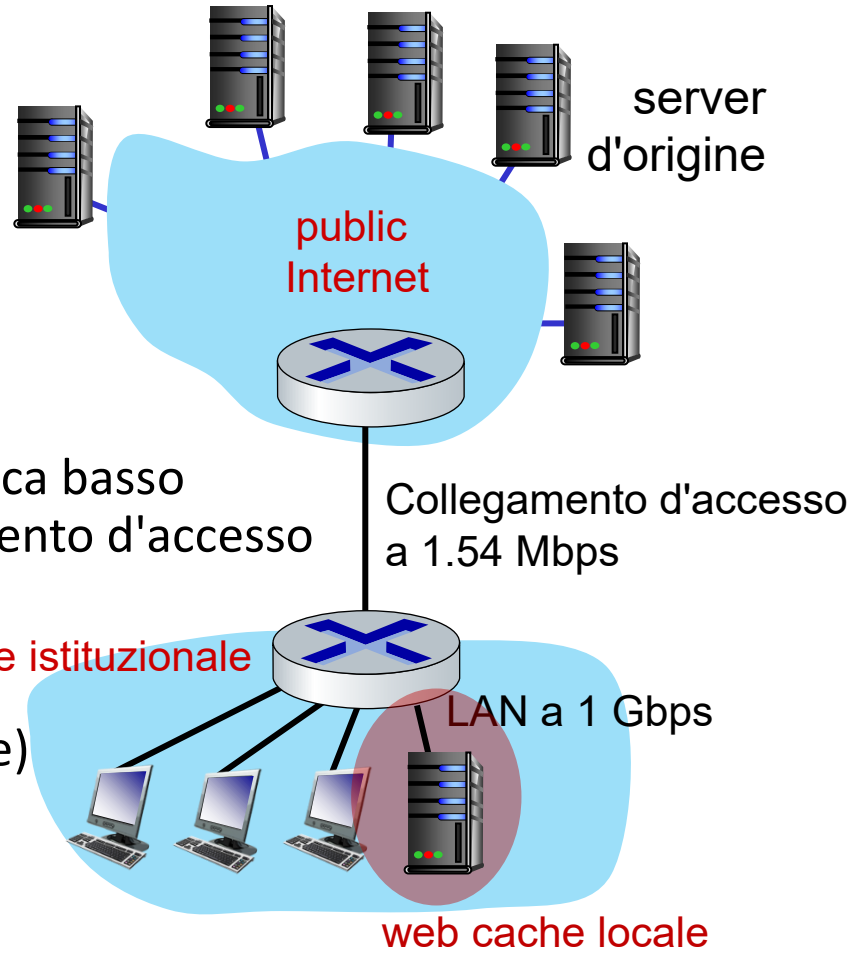
- il 40% delle richieste sarà soddisfatto dalla cache, con ritardo basso (millisecondi)
- 60% delle richieste sarà soddisfatto dal server d'origine
  - tasso di trasmissione sul collegamento d'accesso

    = 0.6 * 1.50 Mbps = .9 Mbps
  - utilizzazione collegamento d'accesso = 0.9/1.54 = .58 significa basso (decine di millisecondi) ritardo di accodamento al collegamento d'accesso
- ritardo end-end medio:

  = 0.6 * (ritardo dai server d'origine)

  + 0.4 * (ritardo quando richiesta soddisfatta dalla cache)

  = 0.6 (2.01) + 0.4 (~ms) = ~ 1.2 s



ritardo medio end-end inferiore che con un collegamento a 154 Mbps (e meno costoso!)

Nota sul caching

Il caching può essere effettuato da:

- una web cache, ossia uno speciale tipo di proxy, cui il browser invia le richieste invece che indirizzarle all'origin server.
- oppure, dal browser stesso, che conserva una copia degli oggetti richiesti in precedenza

In entrambi i casi, occorre prestare attenzione al problema dell'aggiornamento degli oggetti: vedi riga di intestazione *Cache-Control* e *GET condizionale*.

HTTP/2

Obiettivo principale: diminuzione del ritardo nelle richieste HTTP a più oggetti

HTTP1.1: ha introdotto GET multiple in pipeline su una singola connessione TCP

- il server risponde *in ordine* (FCFS: first-come-first-served scheduling) alle richieste GET
- con FCFS, oggetti piccoli possono dover aspettare per la trasmissione (head-of-line (HOL) blocking [blocco in testa alla coda]) dietro a uno o più oggetti grandi
- il recupero delle perdite (ritrasmissione dei segmenti TCP persi) blocca la trasmissione degli oggetti

HTTP/2

Obiettivo principale: diminuzione del ritardo nelle richieste HTTP a più oggetti

HTTP/2: [RFC 7540, 2015] maggiore flessibilità del server nell'invio di oggetti al client:

- metodi, codice di stato, maggior parte dei campi di intestazione inalterati rispetto a HTTP 1.1
- introduce una codifica binaria dei messaggi, con supporto alla compressione degli header
- ordine di trasmissione degli oggetti richiesti basata su una priorità degli oggetti specificata dal client (non necessariamente FCFS)
- invio *push* al client di oggetti aggiuntivi, senza che il client li abbia richiesti
- dividere gli oggetti (in realtà, i messaggi) in frame, alternare i frame relativi a oggetti (messaggi) diversi per mitigare il blocco HOL

HTTP/2

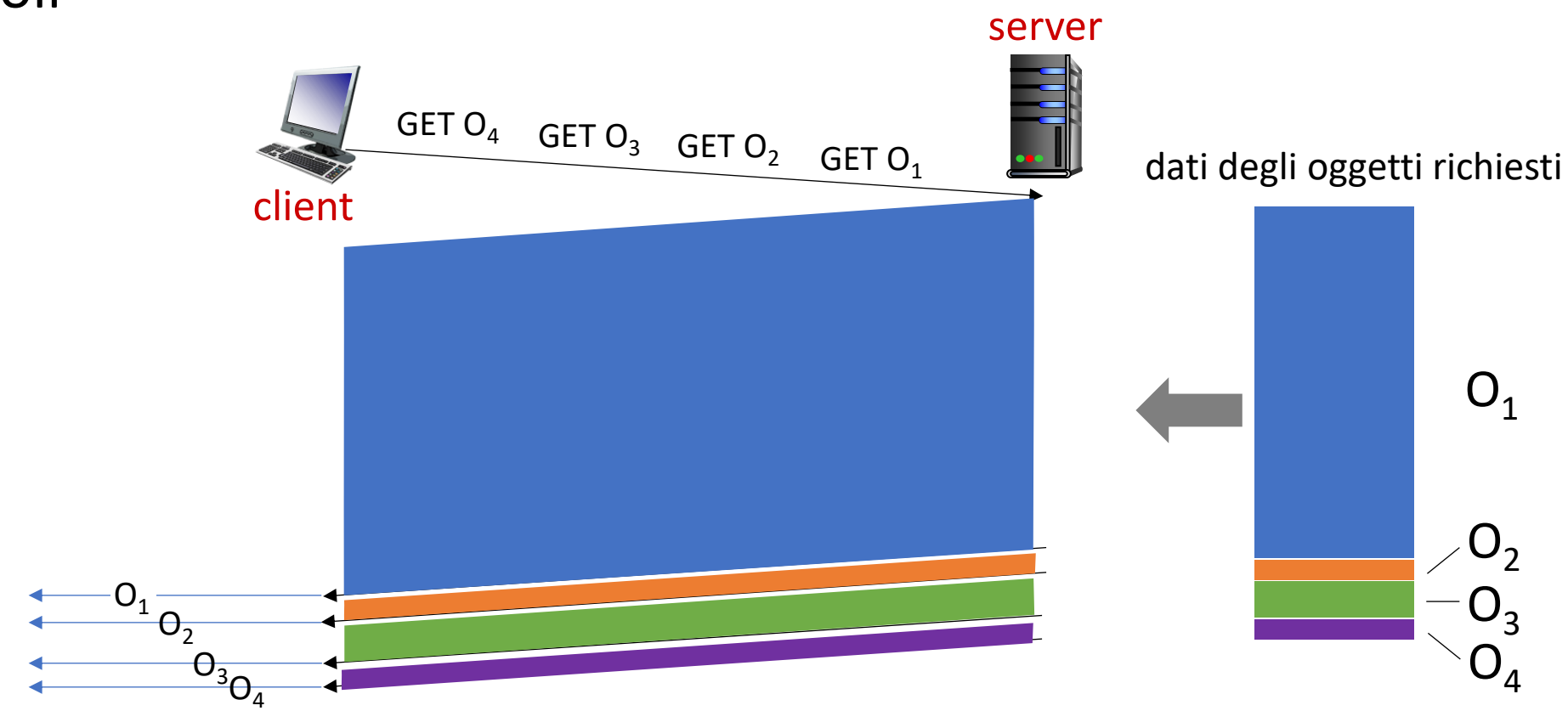
Obiettivo principale: diminuzione del ritardo nelle richieste HTTP a più oggetti

Affrontando l'HOL, HTTP/2 vuole permettere di scaricare una pagina con più oggetti attraverso una singola connessione TCP:

- minor overhead sul server
- miglior funzionamento del controllo della congestione TCP

HTTP/2: mitigazione del blocco HOL

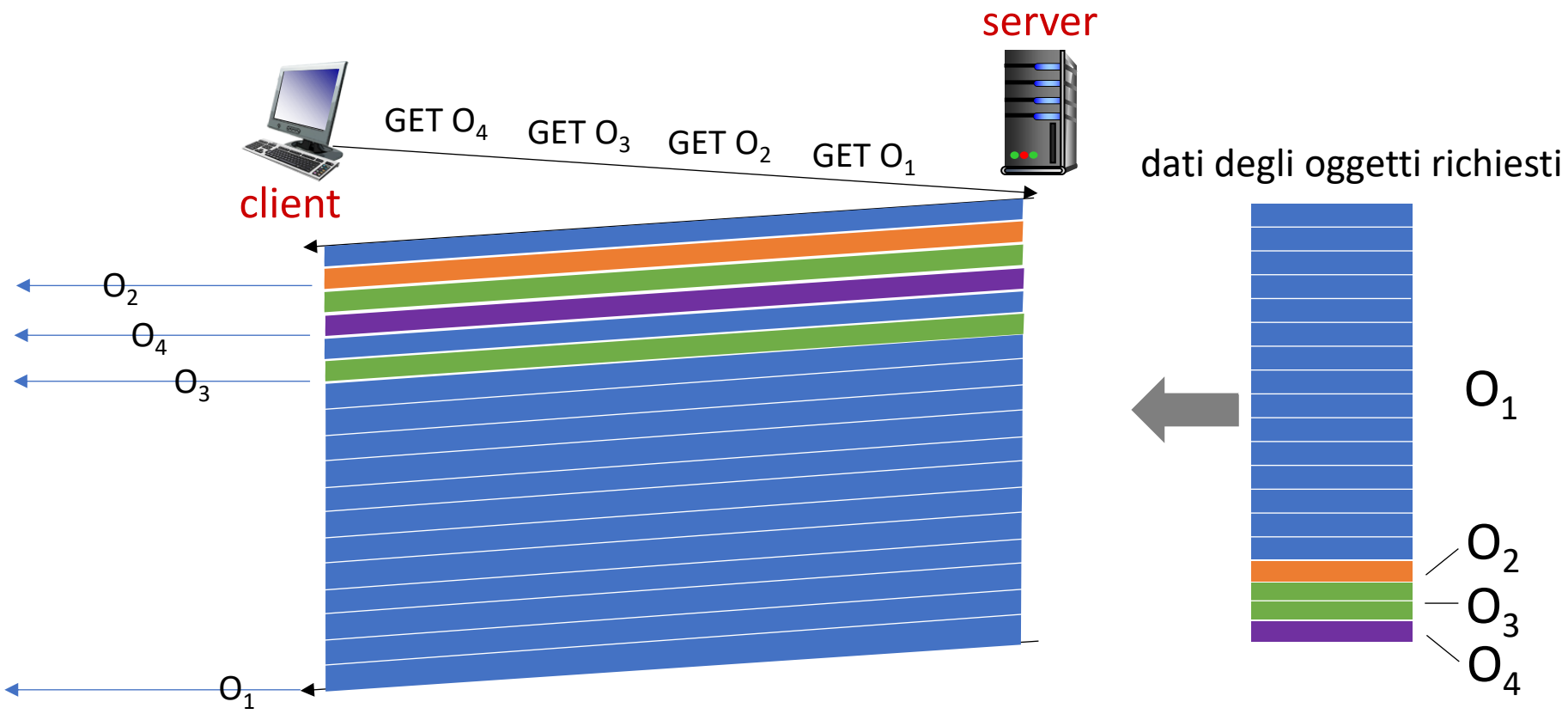
HTTP 1.1: il client richiede 1 oggetto grande (es., file video) e 3 oggetti più piccoli



oggetti consegnati nell'ordine in cui sono stati richiesti: O_2, O_3, O_4 aspettano dietro O_1

mitigazione

HTTP/2: oggetti divisi in frame, trasmissione de frame interlacciata



O_2, O_3, O_4 consegnati rapidamente, O_1 leggermente ritardato

Da HTTP/2 a HTTP/3

HTTP/2 su una singola connessione TCP significa:

- il recupero dalla perdita di pacchetti blocca comunque tutte le trasmissioni di oggetti
  - come in HTTP 1.1, i browser sono incentivati ad aprire più connessioni TCP parallele per ridurre lo stallo e aumentare il throughput complessivo
- nessuna sicurezza su una connessione TCP semplice
- HTTP/3: aggiunge sicurezza, controllo di errore (per oggetto) e congestione (più pipelining) su UDP
  - ulteriori informazioni su HTTP/3 trattando il livello di trasporto